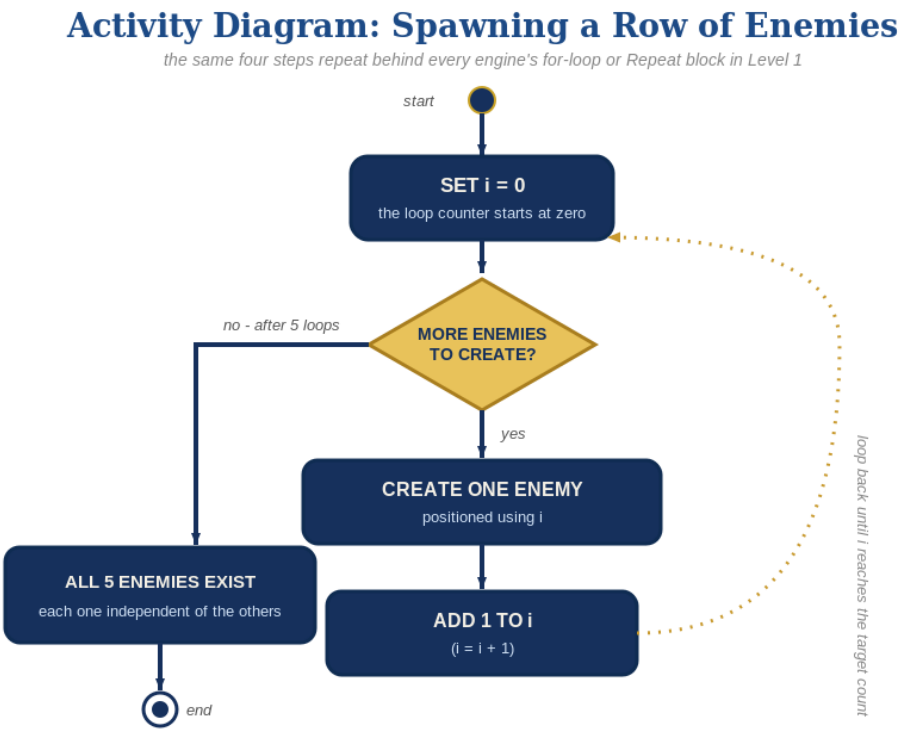
Lesson 5 Teacher's Key

Unity

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution



Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

A new EnemySpawner.cs on a new empty GameObject - Enemy.cs and Bullet.cs need zero changes from Lesson 4:

```
// EnemySpawner.cs:
public GameObject enemyPrefab;
public int enemyCount = 5;

void Start() {
    for (int enemyIndex = 0; enemyIndex < enemyCount; enemyIndex++) {
```

```
        Vector3 pos = new Vector3(-4f + enemyIndex * 2f, 4f, 0f);
        Instantiate(enemyPrefab, pos, Quaternion.identity);
    }
}
```

Level 2 - Complete Worked Solution

Bullet.cs's OnTriggerEnter2D and Enemy.cs's own Start() are fully self-contained per instance, so every prefab copy the loop creates behaves independently and correctly with no extra code needed for Level 2.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: already works

AudioSource.PlayClipAtPoint() in Bullet.cs runs independently for every enemy that gets hit - no changes needed.

Visual: striped row

Set the SpriteRenderer color right after Instantiate() in the spawner, using the loop index. new Color(0.8f, 0.1f, 0.2f) is a deep crimson red and new Color(0.95f, 0.4f, 0.5f) is a lighter dusty rose/pink - alternating them by enemyIndex % 2 makes every other enemy a visibly different shade of red, not a blended new color. Written as a plain if/else rather than a one-line ternary - readable at a glance for a student who hasn't met the ternary operator yet (see this lesson's Book Notes entry: that's deliberately saved for a later, more advanced project):

```
for (int enemyIndex = 0; enemyIndex < enemyCount; enemyIndex++) {
    Vector3 pos = new Vector3(-4f + enemyIndex * 2f, 4f, 0f);
    GameObject newEnemy = Instantiate(enemyPrefab, pos,
Quaternion.identity);
    if (enemyIndex % 2 == 0) {
        newEnemy.GetComponent<SpriteRenderer>().color = new Color(0.8f,
0.1f, 0.2f);
    } else {
        newEnemy.GetComponent<SpriteRenderer>().color = new Color(0.95f,
0.4f, 0.5f);
    }
}
```

Structure: spawner vs. behavior stay separate

EnemySpawner.cs only ever creates and positions enemies - collision-response and hit-sound logic stay entirely inside Enemy.cs and Bullet.cs.

Complete Level 3 solution (all 3 goals combined)

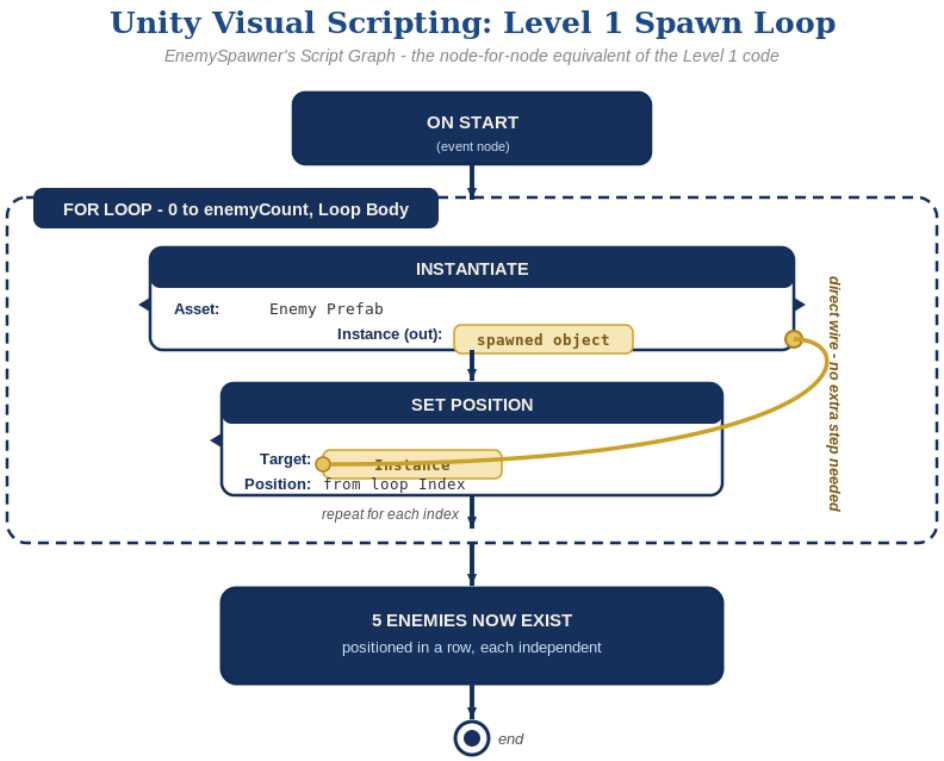
The spawner loop with the visual idea added - sound needs no changes since it already lives on Bullet.cs:

```
public GameObject enemyPrefab;
public int enemyCount = 5;

void Start() {
    for (int enemyIndex = 0; enemyIndex < enemyCount; enemyIndex++) {
```

```
        Vector3 pos = new Vector3(-4f + enemyIndex * 2f, 4f, 0f);
        GameObject newEnemy = Instantiate(enemyPrefab, pos,
Quaternion.identity);
        if (enemyIndex % 2 == 0) {
            newEnemy.GetComponent<SpriteRenderer>().color = new
Color(0.8f, 0.1f, 0.2f);
        } else {
            newEnemy.GetComponent<SpriteRenderer>().color = new
Color(0.95f, 0.4f, 0.5f);
        }
    }
}
```

Alternate Path Notes: Visual Scripting



Instantiate's own output pin already holds the exact instance it just created - dragging that pin straight into Set Position's Target is the normal way node graphs work, not a special trick this lesson had to reach for.

The Script Graph, node by node - Instantiate's own output pin plugs straight into Set Position's Target, no extra step needed.

A For Loop node (0 to enemyCount) feeding Instantiate + Set Position nodes is the graph equivalent of the C# for loop.

The Level 3 goals map onto nodes the same way: a "Set Member" node on the newly spawned instance's Sprite Renderer color for the visual idea - Sound needs no changes either way, since it lives on the Bullet's own graph, not the spawner's.

- Keeping the same Structure principle (a spawner only spawns; behavior lives on the spawned object) whether a student is in C# or graphs reinforces that it's a design idea, not a syntax rule.